

Student ID: 2408861

Student Name: Prachi Rajbanshi

```
pip install keras tensorflow
```

```
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (3.13.2)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras) (0.5.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras) (26.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.78.0)
Requirement already satisfied: tensorboard~2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.1.31)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.19.0->tensorflow) (3.7.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.19.0->tensorflow) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.19.0->tensorflow) (3.1.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~2.19.0->tensorflow) (3.0.2)
```

```
import tensorflow as tf
print(tf.keras.__version__)
```

3.13.2

Gradient Computations with Keras

```
import tensorflow as tf
x = tf.Variable(5.0) # Trainable variable
with tf.GradientTape() as tape:
    y = x ** 2 # y = x^2
grad = tape.gradient(y, x) # Computes dy/dx automatically
print(grad.numpy()) # Output: 10.0
```

10.0

Matrix Multiplication Speed (Numpy vs Tensorflow on GPU)

```
import numpy as np
import tensorflow as tf
import time
# Create large random matrices
size = (1000, 1000)
A = np.random.rand(*size)
B = np.random.rand(*size)
# NumPy Multiplication
start = time.time()
C_numpy = np.dot(A, B)
print("NumPy Time:", time.time() - start)
# TensorFlow Multiplication (for colab uses GPU Runtime if available)
```

```
A_tf = tf.constant(A)
B_tf = tf.constant(B)

start = time.time()
C_tf = tf.matmul(A_tf, B_tf)
print("TensorFlow Time:", time.time()- start)
```

```
NumPy Time: 0.07839107513427734
TensorFlow Time: 0.1708390712738037
```

```
# Implementation of Activation Function with Keras
from tensorflow.keras.layers import Dense
layer = Dense(64, activation='sigmoid')
```

```
# Manually Training Network in Numpy
for epoch in range(10):
    # Forwardpass
    y_pred=np.dot(x_train, weights)
    # Computeloss
    loss=np.mean((y_pred - y_train)**2)
    # Computegradientsmanually
    gradients=2*np.dot(x_train.T,(y_pred - y_train))/len(x_train)
    # Updateweights
    weights-=learning_rate*gradients
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_5812/2754578604.py in <cell line: 0>()
      2 for epoch in range(10):
      3     # Forwardpass
----> 4     y_pred=np.dot(x_train,weights)
      5     # Computeloss
      6     loss=np.mean((y_pred-y_train)**2)

NameError: name 'x_train' is not defined
```

```
# keras training is One Line
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

```
# Layers in Keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(64, activation = 'relu', input_shape=(784,)),
    Dense(10, activation = 'softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Implementing SGD in Numpy
learning_rate = 0.01
weights = np.random.randn(3, 3)
for _ in range(100):
    gradient = np.random.randn(3,3)
    weights -= learning_rate * gradient
```

```
# In Keras
from tensorflow.keras.optimizers import SGD
optimizer = SGD(learning_rate = 0.01)
```

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from PIL import Image #ImportPillow

#Defineddatasetpaths
train_dir = "/content/drive/MyDrive/AIandML/FruitinAmazon/train"
```

```

test_dir = "/content/drive/MyDrive/AIandML/FruitinAmazon/test"

#Defineimagesize
img_height, img_width = 28,28

#FunctiontoloadimagesandlabelsusingPIL
def load_images_from_folder(folder):
    images = []
    labels = []

    class_names = sorted(os.listdir(folder)) # Sortedclassnames(digit_0,digit_1,...)
    class_map = {name: i for i, name in enumerate(class_names)} # Mapclassnamestolabels

    for class_name in class_names:
        class_path = os.path.join(folder, class_name)
        label = class_map[class_name]
        for filename in os.listdir(class_path):
            img_path=os.path.join(class_path, filename)

            #LoadimageusingPIL
            img = Image.open(img_path).convert("L") # Converttograyscale
            img = img.resize((img_width, img_height)) # Resizeto(28,28)
            img = np.array(img)/255.0 # Normalizepixelvaluesto[0,1]

            images.append(img)
            labels.append(label)
    return np.array(images), np.array(labels)

#Load training and testing datasets
x_train, y_train = load_images_from_folder(train_dir)
x_test,y_test = load_images_from_folder(test_dir)

#Reshape images for Keras input
x_train = x_train.reshape(-1, img_height, img_width, 1) # Shape(num_samples,28,28,1)
x_test = x_test.reshape(-1, img_height, img_width, 1)

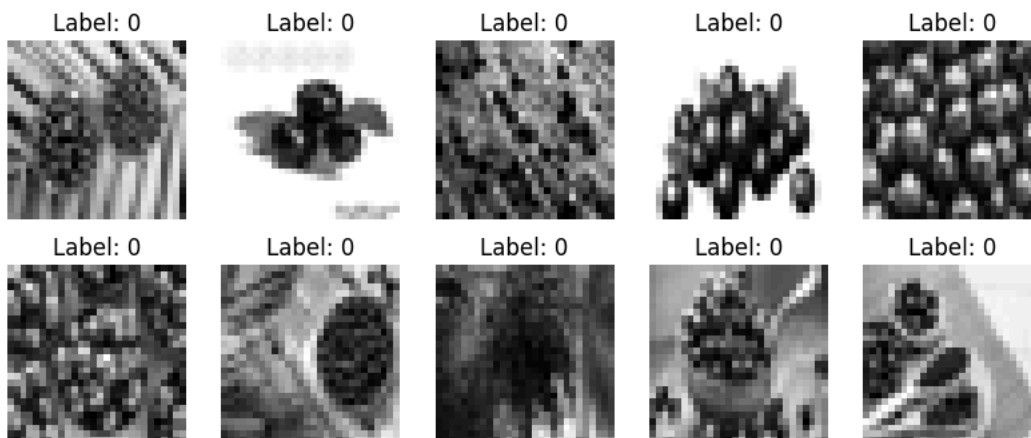
#One-hot encode labels
y_train = to_categorical(y_train, num_classes = 10)
y_test = to_categorical(y_test, num_classes = 10)

#Print dataset shape
print(f"Trainingset: {x_train.shape}, Labels: {y_train.shape}")
print(f"Testing set: {x_test.shape}, Labels: {y_test.shape}")

#Visualize some images
plt.figure(figsize=(10,4))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(x_train[i].reshape(28,28), cmap='gray')
    plt.title(f"Label: {np.argmax(y_train[i])}")
    plt.axis("off")
plt.show()

```

Trainingset: (90, 28, 28, 1), Labels: (90, 10)
 Testing set: (30, 28, 28, 1), Labels: (30, 10)



```
x_train = x_train.reshape(-1, img_height, img_width, 1)
```

```
x_train = x_train.reshape(-1, img_height, img_width, 3)
```

```
# Loading and Preprocessing MNIST Handwritten Digit Dataset
```

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

# Load the MNIST Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize the images to values between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Flatten the 28x28 images into 784-dimensional vectors
x_train = x_train.reshape(-1, 28 * 28)
x_test = x_test.reshape(-1, 28 * 28)

# One-hot encode the labels (0-9) for classification
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

```
# Sequential API
```

```
#Model parameters
```

```
import tensorflow as tf
from tensorflow import keras
num_classes = 10
input_shape = (28,28,1)
model = keras.Sequential(
    [
        keras.layers.Input(shape=input_shape),
        keras.layers.Flatten(), # Flatten the 28x28 image to a 784-dimensional vector
        keras.layers.Dense(64, activation="sigmoid"),
        keras.layers.Dense(128, activation="sigmoid"),
        keras.layers.Dense(256, activation="sigmoid"),
        keras.layers.Dense(num_classes, activation="softmax"),
    ]
)
```

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 64)	50,240
dense_4 (Dense)	(None, 128)	8,320
dense_5 (Dense)	(None, 256)	33,024
dense_6 (Dense)	(None, 10)	2,570

Total params: 94,154 (367.79 KB)

Trainable params: 94,154 (367.79 KB)

Non-trainable params: 0 (0.00 B)

```
# Functional API
```

```
# Model parameters
```

```
import tensorflow as tf
from tensorflow import keras
num_classes = 10
input_shape = (28, 28, 1)
```

```
def build_functional_model():
```

```
    # Input layer
    inputs = keras.Input(shape=input_shape)
    # Flatten layer
    x = keras.layers.Flatten()(inputs)
    # Hidden layers
    x = keras.layers.Dense(64, activation="sigmoid")(x)
    x = keras.layers.Dense(128, activation="sigmoid")(x)
```

```
x = keras.layers.Dense(256, activation="sigmoid")(x)
# Output layer
outputs = keras.layers.Dense(num_classes, activation="softmax")(x)
# Create model
model = keras.Model(inputs=inputs, outputs=outputs)
return model

# Build the model
functional_model = build_functional_model()
functional_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 28, 28, 1)	0
flatten_1 (Flatten)	(None, 784)	0
dense_7 (Dense)	(None, 64)	50,240
dense_8 (Dense)	(None, 128)	8,320
dense_9 (Dense)	(None, 256)	33,024
dense_10 (Dense)	(None, 10)	2,570

Total params: 94,154 (367.79 KB)
Trainable params: 94,154 (367.79 KB)
Non-trainable params: 0 (0.00 B)

⌵

B

I

<>

↺

🖼

”

≡

≡

—

ψ

😊

⋮

Close

=====

****Exercise: Building a Fully Connected Network (FCN) for Devnagari Classification****

=====

=====

Exercise: Building a Fully Connected Network (FCN) for Devnagari Digit Classification

=====

Start coding or [generate](#) with AI.

Task 1: Data Preparation

Loading the Data

- Download the provided folder that contains the Devnagari digits dataset.
- Use the Python Imaging Library (PIL) to load and read the image files from the dataset.
- Convert the images into Numpy arrays and normalize them to a range of 0-1.
- Use train folder for training and test for testing.
- Extract the corresponding labels for each image.

```
# Task 1: Data Preparation
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from PIL import Image
import matplotlib.pyplot as plt

train_dir = "/content/drive/MyDrive/AIandML/DevanagariHandwrittenDigitDataset/Train"
test_dir = "/content/drive/MyDrive/AIandML/DevanagariHandwrittenDigitDataset/Test"
img_height, img_width = 28, 28

def load_images_from_folder(folder):
    images = []
    labels = []
    class_names = sorted(os.listdir(folder))
    class_map = {name: i for i, name in enumerate(class_names)}
```

```

for class_name in class_names:
    class_path = os.path.join(folder, class_name)
    label = class_map[class_name]
    for filename in os.listdir(class_path):
        img_path = os.path.join(class_path, filename)
        img = Image.open(img_path).convert("L") # Convert to Grayscale
        img = img.resize((img_height, img_width)) # Resize to 28 x 28

        # Dividing by 255 - neural networks perform better when input values are between 0 and 1
        img = np.array(img)/255.0 # Normalize
        images.append(img)
        labels.append(label)
    return np.array(images), np.array(labels)

# Load data [cite: 1584]
x_train, y_train = load_images_from_folder(train_dir)
x_test, y_test = load_images_from_folder(test_dir)

```

```

# Load the original integer labels again to reset them
_, y_train = load_images_from_folder(train_dir)
_, y_test = load_images_from_folder(test_dir)

#ONE-HOT ENCODING
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

print(y_train.shape)

```

```
(17000, 10)
```

Task 2 : Build the FCN Model

Model Architecture

- Create a Sequential model using Keras.
- Add 3 hidden layers with the following number of neurons:
 - 1st hidden layer: 64 neurons
 - 2nd hidden layer: 128 neurons
 - 3rd hidden layer: 256 neurons
- Use sigmoid activation functions for all hidden layers.
- Add an output layer with 10 units with softmax (since Devnagari digits have 10 classes) and a softmax activation function.

```

# Task 2 : Build the FCN Model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)), # Flatten 2D to 1D [cite: 1419]
    tf.keras.layers.Flatten(),
    # Sigmoid - For hidden layers
    # Softmax - For final year -> gives porbability distribution across 10 possible digits

    tf.keras.layers.Dense(64, activation='sigmoid'), # 1st Hidden Layer [cite: 1588]
    tf.keras.layers.Dense(128, activation='sigmoid'), # 2nd Hidden Layer [cite: 1588]
    tf.keras.layers.Dense(256, activation='sigmoid'), # 3rd Hidden Layer [cite: 1588]
    tf.keras.layers.Dense(10, activation='softmax') # Output Layer [cite: 1589]
])

```

Task 3 : Compile the Model

Model Compilation

- Choose an appropriate optimizer (e.g., Adam), loss function (e.g., sparse categorical crossentropy), and evaluation metric (e.g., accuracy).

```

# Task 3 : Compile the Model
model.compile(
    optimizer='adam',
    # Categorical Cross-entropy -> Loss function used when labels are one-hot encoded
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
# Train
history = model.fit(x_train, y_train, batch_size=128, epochs=20, validation_split=0.2)
```

Epoch 1/20
107/107 ————— 3s 10ms/step - accuracy: 0.4690 - loss: 1.6200 - val_accuracy: 0.0000e+00 - val_loss: 7.3840
Epoch 2/20
107/107 ————— 1s 7ms/step - accuracy: 0.8465 - loss: 0.5068 - val_accuracy: 0.0000e+00 - val_loss: 9.0604
Epoch 3/20
107/107 ————— 1s 8ms/step - accuracy: 0.9207 - loss: 0.2778 - val_accuracy: 0.0000e+00 - val_loss: 9.5743
Epoch 4/20
107/107 ————— 1s 7ms/step - accuracy: 0.9471 - loss: 0.1913 - val_accuracy: 0.0000e+00 - val_loss: 9.8576
Epoch 5/20
107/107 ————— 1s 7ms/step - accuracy: 0.9571 - loss: 0.1491 - val_accuracy: 0.0000e+00 - val_loss: 10.2100
Epoch 6/20
107/107 ————— 1s 7ms/step - accuracy: 0.9654 - loss: 0.1178 - val_accuracy: 0.0000e+00 - val_loss: 10.4409
Epoch 7/20
107/107 ————— 1s 11ms/step - accuracy: 0.9736 - loss: 0.0953 - val_accuracy: 0.0000e+00 - val_loss: 10.7394
Epoch 8/20
107/107 ————— 1s 14ms/step - accuracy: 0.9779 - loss: 0.0783 - val_accuracy: 0.0000e+00 - val_loss: 10.5150
Epoch 9/20
107/107 ————— 1s 12ms/step - accuracy: 0.9821 - loss: 0.0659 - val_accuracy: 0.0000e+00 - val_loss: 11.0371
Epoch 10/20
107/107 ————— 1s 13ms/step - accuracy: 0.9859 - loss: 0.0548 - val_accuracy: 0.0000e+00 - val_loss: 11.1251
Epoch 11/20
107/107 ————— 2s 7ms/step - accuracy: 0.9885 - loss: 0.0455 - val_accuracy: 0.0000e+00 - val_loss: 11.2878
Epoch 12/20
107/107 ————— 1s 7ms/step - accuracy: 0.9917 - loss: 0.0373 - val_accuracy: 0.0000e+00 - val_loss: 11.4645
Epoch 13/20
107/107 ————— 1s 7ms/step - accuracy: 0.9930 - loss: 0.0330 - val_accuracy: 0.0000e+00 - val_loss: 11.6547
Epoch 14/20
107/107 ————— 1s 7ms/step - accuracy: 0.9940 - loss: 0.0269 - val_accuracy: 0.0000e+00 - val_loss: 11.8419
Epoch 15/20
107/107 ————— 1s 7ms/step - accuracy: 0.9959 - loss: 0.0225 - val_accuracy: 0.0000e+00 - val_loss: 12.0398
Epoch 16/20
107/107 ————— 1s 7ms/step - accuracy: 0.9957 - loss: 0.0200 - val_accuracy: 0.0000e+00 - val_loss: 12.0135
Epoch 17/20
107/107 ————— 1s 7ms/step - accuracy: 0.9971 - loss: 0.0162 - val_accuracy: 0.0000e+00 - val_loss: 12.0802
Epoch 18/20
107/107 ————— 1s 7ms/step - accuracy: 0.9982 - loss: 0.0129 - val_accuracy: 0.0000e+00 - val_loss: 12.2794
Epoch 19/20
107/107 ————— 1s 7ms/step - accuracy: 0.9989 - loss: 0.0104 - val_accuracy: 0.0000e+00 - val_loss: 12.3214
Epoch 20/20
107/107 ————— 1s 7ms/step - accuracy: 0.9990 - loss: 0.0092 - val_accuracy: 0.0000e+00 - val_loss: 12.6049

Task 4: Train the Model

Model Training

- Use the `model.fit()` function to train the model. Set the batch size to 128 and the number of epochs to 20.
- Use validation split (validation split=0.2) to monitor the model's performance on validation data.
- Optionally, use callbacks such as `ModelCheckpoint` and `EarlyStopping` for saving the best model and avoiding overfitting.

```
# Task 4 : Train the Model

history = model.fit(
    x_train, y_train,
    batch_size=128,
    epochs=20,
    validation_split=0.2
)
```

Epoch 1/20
107/107 ————— 1s 7ms/step - accuracy: 0.9993 - loss: 0.0077 - val_accuracy: 0.0000e+00 - val_loss: 12.6410
Epoch 2/20
107/107 ————— 1s 7ms/step - accuracy: 0.9995 - loss: 0.0058 - val_accuracy: 0.0000e+00 - val_loss: 12.7866
Epoch 3/20
107/107 ————— 1s 7ms/step - accuracy: 0.9996 - loss: 0.0050 - val_accuracy: 0.0000e+00 - val_loss: 13.0436
Epoch 4/20
107/107 ————— 1s 7ms/step - accuracy: 0.9996 - loss: 0.0040 - val_accuracy: 0.0000e+00 - val_loss: 13.0357
Epoch 5/20
107/107 ————— 1s 7ms/step - accuracy: 0.9998 - loss: 0.0035 - val_accuracy: 0.0000e+00 - val_loss: 13.3161

Task 5 : Evaluate the Model

Model Evaluation

- After training, evaluate the model using `model.evaluate()` on the test set to check the test accuracy and loss.

```
# Visualization (Requirement from Deliverables)
```

```

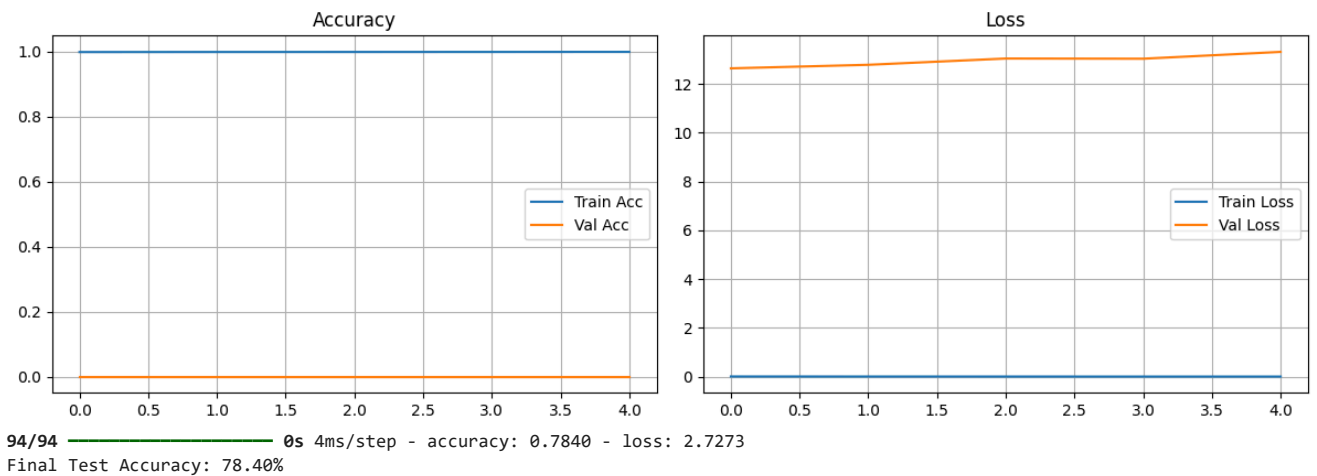
# visualization (requirement from deliverables)
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.legend(); plt.title('Accuracy')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend(); plt.title('Loss')
plt.grid(True)

plt.tight_layout()
plt.show()

# Task 5: Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Final Test Accuracy: {test_acc*100:.2f}%")

```



Task 6 : Save and Load the Model

Model Saving and Loading

- Save the trained model to an .h5 file using `model.save()`.
- Load the saved model and re-evaluate its performance on the test set

```

# Task 6 : Save and Load the Model
# Save [cite: 1565]
model.save('devnagari_model.h5')

# Load and Re-evaluate [cite: 1569, 1571]
loaded_model = tf.keras.models.load_model('devnagari_model.h5')
loaded_model.evaluate(x_test, y_test)

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until
94/94 ————— 1s 3ms/step - accuracy: 0.7840 - loss: 2.7273
[2.7273032665252686, 0.783999794960022]

Task 7: Predictions

Making Predictions

- Use `model.predict()` to make predictions on test images.
- Convert the model's predicted probabilities to digit labels using `np.argmax()`

```

# Task 7 : Predictions
predictions = model.predict(x_test)

```



```
# Convert probabilities to class indices [cite: 1559, 1560]
predicted_labels = np.argmax(predictions, axis=1)

# Display first prediction result
print(f"Predicted: {predicted_labels[0]}, True: {np.argmax(y_test[0])}")
```

94/94 ————— 0s 4ms/step
 Predicted: 0, True: 0

Task : Visualization

```
import matplotlib.pyplot as plt

# Check the available keys in history
# print(history.history.keys())

# Plot Training & Validation Accuracy
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', color='blue', linestyle='--')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='red')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Plot Training & Validation Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', color='blue', linestyle='--')
plt.plot(history.history['val_loss'], label='Validation Loss', color='red')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

